

SPÉCIFICATION DÉTAILLÉE ALG_CONSENSUS — Consensus d'état d'essaim par diffusion du maximum

Simulateur déterministe — version 7 (Rust v6)

Fiche documentaire

Champ	Valeur
Identifiant de l'algorithme	ALG_CONSENSUS
Nom de l'algorithme	Consensus d'état d'essaim par diffusion du maximum
Référence document	SWARM-SPEC-ALG_CONSENSUS-v7
Version	7.0
Statut	VALIDÉ — spécification v7 (canal unique, checkpoints exacts, moteur protocole_v6::Moteur)
Classification	Interne
Rédacteur	Hermès — dérivé de la spécification normative du dépôt alg-consensus (branche v6)
Date	2026-09-28
Source de vérité	SPECIFICATION_V7.md, branche v6 de dagornc/alg-consensus

1. Introduction

1.1 Objet du document

Cette spécification définit le contrat du moteur de consensus v7, implémenté en Rust sous protocole_v6::Moteur. Elle prolonge la spécification v6 sans la remplacer rétroactivement : les moteurs Rust v1-v5 restent disponibles avec leur contrat historique.

1.2 Périmètre

Cette version définit un simulateur déterministe de diffusion du maximum.

Elle ne définit pas un protocole de consensus byzantin, ni Raft, ni un transport réseau déployable. Toute lecture qui étendrait ce périmètre est explicitement hors contrat.

1.3 Résumé exécutif

Le moteur fait converger 30 agents sur une grille 6×5 vers le maximum lexicographique d'un couple (valeur, horloge). La fusion est associative, commutative et idempotente. Un canal unique ordonne strictement les émissions, les livraisons et les compteurs, ce qui rend l'exécution reproductible et la reprise exacte. Le mode agrégé réduit le coût de stockage d'un facteur mesuré de $7,1 \times$ sans modifier une seule décision d'admission ni un seul tirage aléatoire.

1.4 Principes de rédaction

Chaque affirmation de ce document est soit un fait vérifié dans le code source, soit une limite explicite. Aucune garantie n'est revendiquée au-delà de ce que les tests et les mesures établissent.

2. Contexte de l'algorithme

2.1 Place dans l'architecture de l'essaim

ALG_CONSENSUS fournit la couche de décision partagée : il permet à l'essaim de maintenir un état commun sans coordinateur central. Il est consommé par les algorithmes d'allocation de tâches et de contrôle de formation.

2.2 Applicabilité

Le moteur s'applique à un essaim de 30 identifiants fixes disposés sur une grille 6×5 à voisinage orthogonal. Ce cadre est celui du simulateur de référence ; il ne préjuge pas d'un déploiement sur une topologie réelle.

2.3 Hypothèses d'exécution

- Un agent est soit inactif (None), soit porteur d'un couple (valeur: i32, horloge: u32).

- La fusion prend le maximum lexicographique du couple complet, sans incrément automatique de l'horloge.
- Entre événements explicites, l'état d'un agent actif ne décroît pas.
- Une réintégration peut injecter un état plus petit ; cette exception est intentionnelle et versionne son incarnation.

2.4 Contraintes et interfaces

L'accord exige au moins un actif et l'égalité de tous les couples actifs.

C'est une observation centralisée, pas une preuve de terminaison distribuée.

3. Modèle mathématique

3.1 Variables et notation

Symbole	Signification
N	Nombre d'agents (30)
G	Grille 6 × 5, voisinage orthogonal
s_i	État de l'agent i : None ou (v, h)
⊔	Fusion : maximum lexicographique du couple
T	Tour courant
λ	Latence de livraison (tours)

3.2 Description formelle

La fusion $\sqcup$ est définie par :

$$(a, ha) \sqcup (b, hb) = (a, ha) \quad \text{si } (a, ha) \geq_{\text{lex}} (b, hb) \\ = (b, hb) \quad \text{sinon}$$

Propriétés. $\sqcup$ est associative, commutative et idempotente. Ces trois propriétés sont la base de l'équivalence entre fusion successive et fusion unique utilisée par le mode agrégé.

3.3 Propriétés et théorèmes

- Monotonie. Entre événements explicites, $s_i(T+1) \geq_{\text{lex}} s_i(T)$ pour tout agent actif i.
- Convergence conditionnelle. Sous population finalement stable, absence de saturation, communications récurrentes équitables et exécution illimitée, chaque actif peut recevoir le maximum survivant via les sondes.

- Limite. Cela ne garantit pas la convergence dans un horizon ou budget fini, ni avec perte totale, ni la conservation d'un maximum dont le dernier porteur disparaît avant toute transmission.

4. Algorithme détaillé

4.1 Ordre d'une période (canal unique)

1. Incrémenter le tour, puis appliquer les événements de ce tour dans l'ordre des identifiants : retrait ou injection d'une nouvelle incarnation.
2. Livrer les messages échus, en rejetant les incarnations obsolètes.
3. Photographier les états qui seront émis. Parcourir les sources puis les voisins dans l'ordre déterministe de `sim::voisins`, puis l'éventuelle sonde.
4. Chaque copie consomme une tentative et un tirage SplitMix64, ****même rejetée****. Classer par priorité : partition, destination inactive, perte aléatoire, saturation, admission. Une probabilité de perte de 1 bloque donc aussi les sondes : aucune lecture ou fusion distante ne contourne le canal.
5. Livrer les messages de latence zéro après toutes les émissions : pas de cascade instantanée dépendant de l'ordre des identifiants.
6. Actualiser les compteurs de stabilité et publier l'observation.

4.2 Latence et partition

Une émission au tour t arrive au tour $t + \text{latence}$. La partition sépare les colonnes 0-2 des colonnes 3-5 et bloque les admissions jusqu'à `fin_partition` inclus. Les conditions du lien sont testées à l'admission ; un retrait de la source n'annule pas un message déjà admis. Le changement d'incarnation de la destination, lui, l'invalidé à la livraison.

4.3 Quiescence et reconnexion

Un actif émet vers ses voisins si la quiescence est désactivée, s'il vient de changer par livraison différée, si sa stabilité est sous le seuil, ou lors d'un réveil périodique. Indépendamment, tous les sonde tours, il émet vers

(id + offset) mod 30, offset parcourant 1...29. Une cible déjà voisine n'est pas dupliquée. Zéro désactive les sondes. **Aucun pair n'est sélectionné par lecture préalable de son état ou de son activité.**

4.4 Optimisation du calendrier (mode agrégé)

Le mode agrégé conserve, pour chaque date d'arrivée et destination, un seul couple maximal, l'incarnation cible et le nombre de copies admises. À latence fixe et événements uniquement aux frontières des tours, les messages d'une même

case visent la même incarnation. L'associativité du max permet de remplacer leurs fusions successives par une fusion unique.

Invariant critique. Les décisions d'admission et les tirages ne sont jamais agrégés : ordre RNG, pertes et capacité restent identiques au mode brut. Cette équivalence ne s'étend pas sans nouvelle preuve aux délais variables, effets secondaires par message ou fusions non idempotentes.

5. Interfaces et données

5.1 Comptabilité

tentatives = perdues + partition + inactif + saturation + livrees + incarnation + en_vol

La capacité est mesurée en copies logiques, identique dans les deux modes.

Même à latence zéro, l'admission précède la livraison et peut saturer.

5.2 Arrêt

Budget épuisé : arrêt des émissions au message exact, fin de la période courante, puis arrêt du moteur sans vidange forcée. À l'horizon, les messages futurs restent en vol. L'accord ne provoque pas d'arrêt anticipé : les événements futurs sont honorés.

5.3 Validation des entrées

Paramètre	Borne
Horizon	≤ 100 000
Latence	≤ 10 000

Duplication	≤ 64
Budget	≤ 10 millions
Capacité	$\leq 100\,000$
Événements	$\leq 10\,000$
Probabilité	finie dans $[0, 1]$
Réveil	non nul
Partition	dans l'horizon

Les événements sont strictement ordonnés (tour, agent), tours 1...horizon, identifiants 0...29. Budget, horizon, duplication et capacité nuls sont autorisés et peuvent empêcher toute communication. Une population vide n'est pas déclarée en accord.

5.4 Reprise exacte

Checkpoint JSON format 1 : configuration, états, tour, RNG, stabilité, incarnations, calendrier complet, compteurs et mode de stockage. Sérialisation des flottants avec aller-retour exact. Reprise aux frontières des périodes, sans reseeding et sans modification implicite des paramètres.

Chargement limité à 64 Mio ; schéma strict, limites de configuration, conservation des compteurs, incarnations attendues, calendrier et sommes des copies contrôlés. Ce contrôle structurel ne prouve pas l'authenticité ni l'histoire d'un fichier forgé. Aucun secret n'est nécessaire ou enregistré. Le CLI refuse d'écraser un fichier existant et synchronise le fichier écrit. Il ne promet pas une transaction durable après panne du système de fichiers : une écriture interrompue peut laisser un fichier incomplet, rejeté à la reprise.

6. Validation et tests

6.1 Résultats mesurés

Test	Résultat
Tests unitaires et d'intégration protocole_v6	10/10
Tests optimise_v5	6/6
Parité Rust / Python	900 graines, 0 écart — bit-à-bit conforme

6.2 Mesures de performance

Mode	Quiescence	Durée (ms)	Tentatives	Accords	Pic de lots
Oracle brut	non	685,702	8 381 200	100	2 468
Agrégé	non	96,104	8 381 200	100	7
Oracle brut	oui	181,465	2 131 640	100	2 426
Agrégé	oui	39,190	2 131 640	100	7

Lecture. Le mode agrégé est $7,1 \times$ plus rapide à tentatives identiques (8 381 200 dans les deux modes) : l'équivalence fonctionnelle est préservée, le gain porte uniquement sur le coût de stockage. Le pic de lots passe de 2 468 à 7. Le mode brut est un oracle de stockage volontairement simple, pas une baseline optimisée.

6.3 Portée de la validation

Les tests différentiels comparent le stockage brut et agrégé à chaque tour ; ils partagent le protocole, donc des cas adverses indépendants sont indispensables. Leur succès constitue une validation expérimentale bornée, jamais une preuve d'absence universelle de défauts.

7. Annexes

7.1 Références documentaires

Réf.	Document / Source
D1	SPECIFICATION_V7.md — spécification normative, branche v6 de dagornc/alg-consensus
D2	V6_PROTOCOL.md — développement Rust v6, contrat et API
D3	AUDIT_V6.md — audit contradictoire Rust v6 / spécification v7
D4	SPECIFICATION_V6.md — spécification v6 (version antérieure)
D5	V5_OPTIMISATION.md — optimisation du calendrier (mode agrégé)

7.2 Références scientifiques

Réf.	Auteurs	Titre	Source	DOI
S1	Shapiro, Preguiça, Baquero, Zawirski	Conflict-free Replicated Data Types	SSS 2011	10.1007/978-3-642-24550-3_29
S2	Demers et al.	Epidemic Algorithms for Replicated Database Maintenance	PODC 1987	10.1145/41840.41841
S3	Vogels	Eventually Consistent	CACM 2009	10.1145/1435417.1435432
S4	Lamport	Time, Clocks, and the Ordering of Events in a Distributed System	CACM 1978	10.1145/359545.359563

7.3 Limites connues

- Simulateur déterministe, pas un protocole byzantin ni un transport réel.
- Le PRNG fini n'est pas une preuve d'indépendance probabiliste ni une source cryptographique.
- La convergence n'est garantie que sous les conditions de §3.3.
- L'équivalence brut/agrégé ne s'étend pas aux délais variables sans preuve.
- Le contrôle de checkpoint ne protège pas contre un fichier forgé.

7.4 Licence

MIT — Copyright (c) 2026 Christophe Dagorn.